OFFICIAL USER MANUAL

PixiVFX Weaver

AI-assisted slot symbol VFX editor for PixiJS. Create, preview, tune, and export particle effects for casino game symbols.

Version: 1.0
Stack: React · Vite · TypeScript · PixiJS v7 · @pixi/particle-emitter
Repository: github.com/stefanskytom/vfx_editor

Table of Contents

1. Getting Started with Cursor

This section explains how to clone the repository, open it in Cursor, install dependencies, and run the development server.

1.1 Prerequisites

- **Cursor IDE** — download from `cursor.com`
- **Node.js** — version 18 or newer (20+ recommended)
- **Git** — for cloning the repository
- **GitHub access** — read access to the repository (write access if you plan to push changes)

1.2 Clone the Repository

Open a terminal and clone the project:

```
git clone https://github.com/stefanskytom/vfx_editor.git
cd vfx_editor
```

Alternatively, use SSH if your GitHub key is configured:

```
git clone git@github.com:stefanskytom/vfx_editor.git
cd vfx_editor
```

1.3 Open the Project in Cursor

1. Launch **Cursor**.
2. Go to **File → Open Folder...** (macOS: **Cursor → Open...**).
3. Select the cloned `vfx_editor` folder.
4. Cursor will index the project and detect it as a Vite + React + TypeScript app.

> **Tip:** You can also open the folder from the terminal with `cursor .` if the Cursor CLI is installed.

1.4 Install Dependencies

Open Cursor's integrated terminal (**View → Terminal** or `Ctrl+`` / `Cmd+``) and run:

```
npm install
```

1.5 Start the Development Server

```
npm run dev
```

The app runs on a fixed port:

```
http://localhost:5176/
```

If the port is busy or you see stale UI after updates, restart with:

```
npm run dev:restart
```

1.6 Production Build (Optional)

```
npm run build
npm run preview
```

1.7 Useful npm Scripts

Command	Description
`npm run dev`	Start Vite dev server on port 5176
`npm run dev:restart`	Kill port 5176 and restart dev server
`npm run build`	Type-check and build production bundle
`npm run lint`	Run Oxlint on source files
`npm run manual:build`	Regenerate this PDF manual with screenshots

2. Workspace Overview

PixiVFX Weaver is organized into three main columns. The left column handles symbol input, AI analysis logs, win states, and export. The center column shows the live PixiJS preview, performance stats, and timeline. The right column contains all tuning controls.

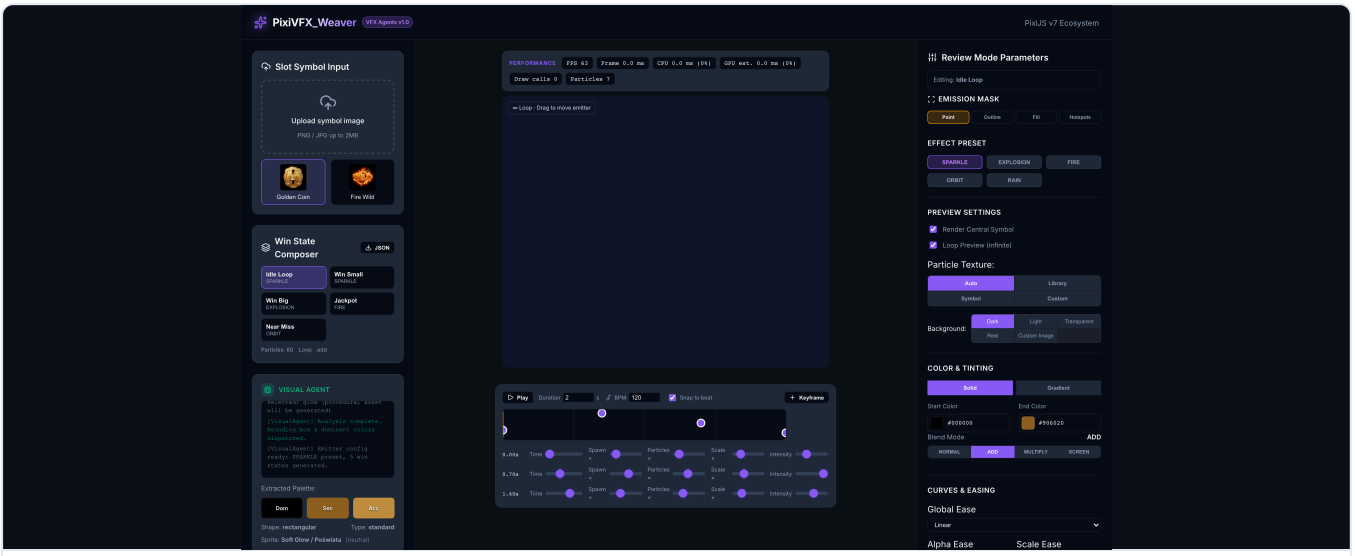

Figure 1 — Full workspace: left input/export panel, center preview & timeline, right parameter controls.

Left Panel

- Symbol upload & presets
- Win State Composer
- Visual Agent analysis log

Center + Right

- Live particle preview (PixiJS)
- Performance monitor
- Timeline & beat sync

- Export & Download
- All emitter tuning sliders

3. Left Panel — Input & Export

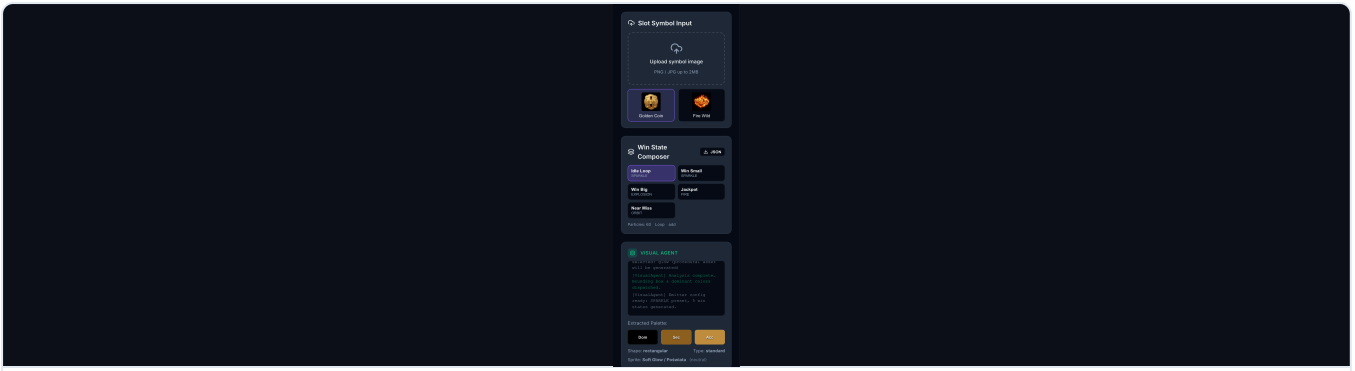

Figure 2 — Left panel: symbol input, win states, Visual Agent, and export section.

3.1 Slot Symbol Input

Upload a PNG or JPG symbol (up to ~2 MB) or pick a built-in preset:

- **Golden Coin** — gold-themed analysis preset
- **Fire Wild** — fire-themed analysis preset

When you upload or select a symbol, the **Visual Agent** analyzes colors, shape, brightness, and VFX theme, then auto-generates emitter parameters and a 5-state win pack.

3.2 Win State Composer

After analysis, five win states are generated automatically:

State	Typical Use	Default Preset
Idle Loop	Symbol at rest on the reel	Sparkle (subtle)
Win Small	Minor line win	Sparkle (medium)
Win Big	Large win burst	Explosion (one-shot)
Jackpot	Top win celebration	Fire (upward)
Near Miss	Almost-win tease	Orbit (screen blend)

Click a state to load its parameters into Review Mode. Edits you make are saved back to the active state. Use the **JSON** button for a quick win-states JSON download.

3.3 Visual Agent

Displays real-time analysis logs and extracted palette swatches (Dominant, Secondary, Accent). Also shows detected shape, symbol type, procedural sprite choice, and inferred VFX theme.

The Visual Agent drives automatic preset selection — you can override everything manually in the right panel afterward.

4. Center Panel — Preview & Timeline

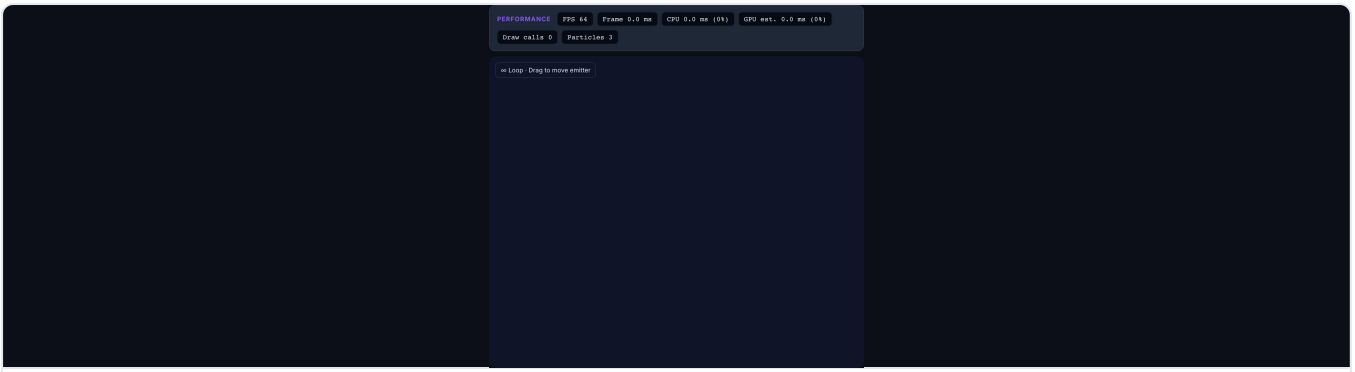

Figure 3 — Live preview with performance stats and emitter HUD.

4.1 Performance Bar

Displayed above the canvas:

- **FPS** — frames per second (warns below 50)
- **Frame ms** — total frame time
- **CPU / GPU est.** — estimated update vs render cost
- **Draw calls** — renderer batch count
- **Particles** — live particle count

4.2 Canvas Interaction

- **Drag on canvas** — move the emitter spawn point (and symbol if visible)
- **Custom background mode** — drag the transform box to pan; drag corners to scale
- **HUD label** — shows loop/burst mode, timeline playback, mask mode, background edit hint

4.3 Emission Mask Overlay

When a mask mode other than *Point* is active, colored dots show spawn regions relative to the symbol. Toggle overlay visibility in the right panel under **Emission Mask**.

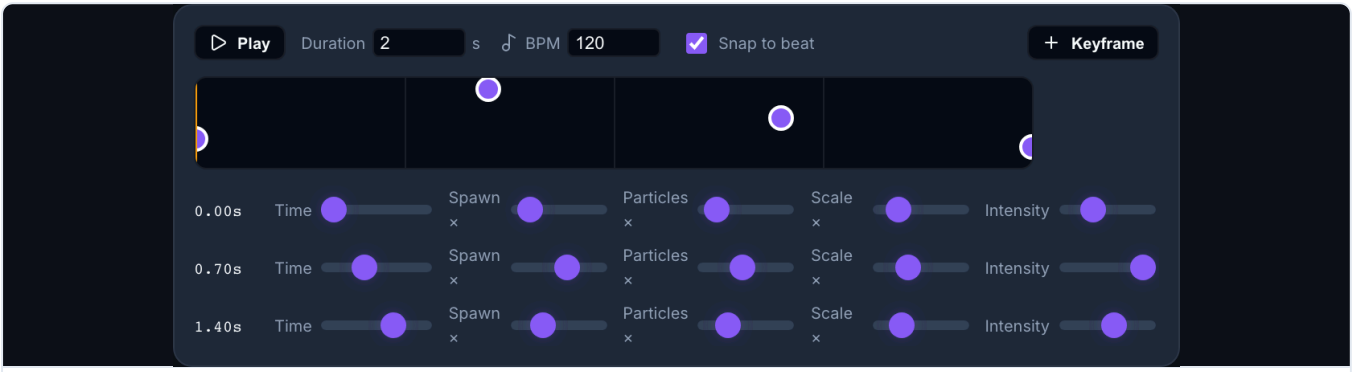

Figure 4 — Timeline editor with beat grid, keyframes, and per-keyframe modifiers.

4.4 Timeline & Beat Sync

Animate spawn intensity over time for win sequences synced to music or game events.

Control	Description
Play / Pause	Preview timeline-driven modulation in the canvas
Duration	Total scene length in seconds (0.5–10 s)
BPM	Beats per minute for grid and snap (60–200)
Snap to beat	Quantize keyframe placement to beat grid
Keyframe	Add keyframe at current playhead (max 8)
Drag keyframe	Move horizontally on the track
Scrub track	Click/drag empty track area to move playhead

Per-Keyframe Modifiers

- **Time** — normalized position (0–1) along timeline
- **Spawn ×** — spawn rate multiplier
- **Particles ×** — max particles multiplier
- **Scale ×** — particle container scale multiplier
- **Intensity** — overall opacity multiplier

5. Right Panel — Review Parameters

All controls apply to the **currently selected win state**. The badge at the top shows which state you are editing.

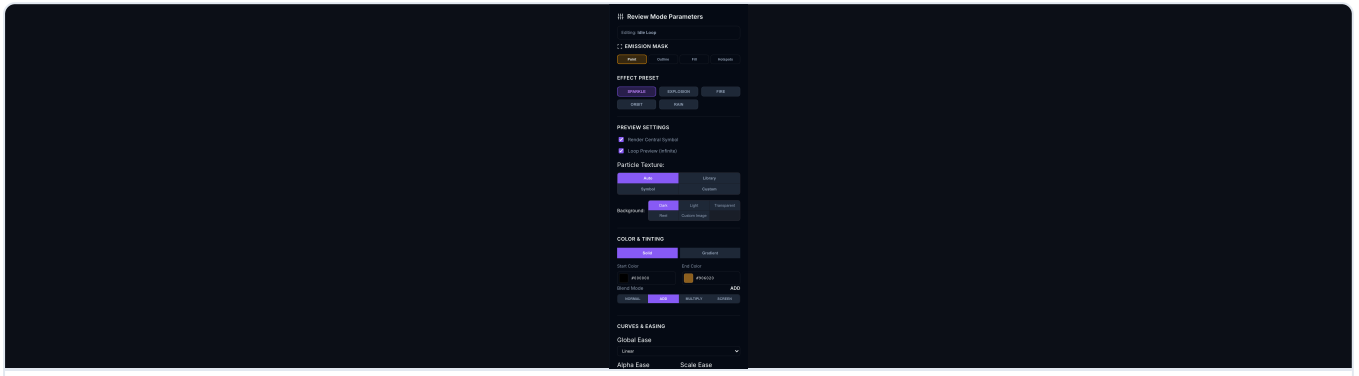

Figure 5 — Emission mask, presets, and preview settings.

5.1 Emission Mask

Mode	Behavior
Point	Classic center emitter (default)
Outline	Particles spawn from symbol edges
Fill	Particles spawn inside symbol body

Mode	Behavior
Hotspots	Spawn from brightest accent regions

Toggle **Show mask overlay** to visualize spawn points on the canvas.

5.2 Effect Preset

Quick-switch between five base behaviors. Changing preset also updates the procedural sprite type:

- **SPARKLE** — gentle additive sparks
- **EXPLOSION** — fast radial burst
- **FIRE** — upward flame with negative gravity option
- **ORBIT** — torus spawn, screen blend
- **RAIN** — downward particles from a rectangle

5.3 Preview Settings

Option	Description
Render Central Symbol	Show/hide the slot symbol sprite on canvas
Loop Preview	Infinite emission vs timed burst
Burst Duration	Emitter lifetime when loop is off (0.2–5 s)
Particle Texture	Auto (procedural) · Library · Symbol · Custom upload
Background	dark · light · transparent · reel · custom image

Particle Texture Sources

- **Auto** — procedural canvas sprite based on analysis (spark, flame, ember, etc.)
- **Library** — 80 bundled PNG textures with search & category filter
- **Symbol** — reuse the slot symbol image as particle texture
- **Custom** — upload PNG/JPG/WebP sprite

Custom Background

Upload an image, then use **Fit** or **Cover** presets. Drag the labeled box to reposition; drag corner handles to scale proportionally.

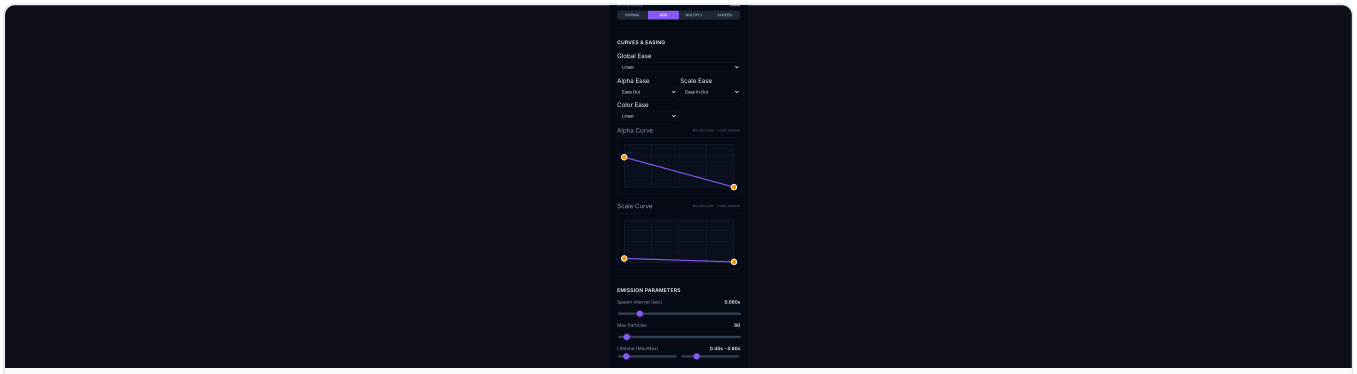

Figure 6 — Color & tinting, curves, and easing controls.

5.4 Color & Tinting

- **Solid** — start/end color pickers (synced to color curve endpoints)
- **Gradient** — multi-stop gradient editor (up to 8 stops); drag middle stops, add/remove stops
- **Blend Mode** — normal · add · multiply · screen

5.5 Curves & Easing

Fine-grained control over particle lifetime animation:

- **Global Ease** — emitter-level easing
- **Alpha / Scale / Color Ease** — per-property easing
- **Alpha Curve** — opacity over lifetime (double-click to add point, right-click to remove)
- **Scale Curve** — size over lifetime

Figure 7 — Emission, motion, scale, and opacity parameters.

5.6 Emission Parameters

- **Spawn Interval** — seconds between particle spawns (0.001–0.5)
- **Max Particles** — pool limit (10–1000)
- **Lifetime Min/Max** — particle lifespan range in seconds

5.7 Motion & Physics

- **Initial Speed Min/Max** — starting velocity in px/s
- **Gravity/Drift Y** — vertical acceleration (−500 to +800 px/s²)
- **Wind/Drift X** — horizontal acceleration

5.8 Scale & Opacity

- **Scale Start / End** — synced to scale curve endpoints
- **Opacity Start / End** — synced to alpha curve endpoints

6. Export & Download

The export section is located at the bottom of the **left panel**.

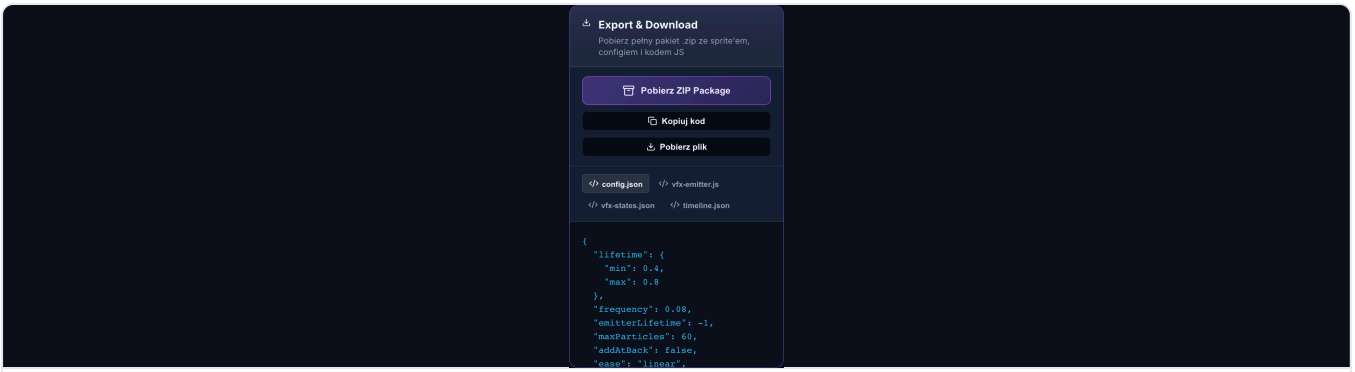

Figure 8 — Export & Download panel with ZIP, copy, and file download actions.

6.1 Primary Actions

Button	Output
Pobierz ZIP Package	Full archive: configs, JS boilerplate, particle sprite, symbol image
Kopiuj kod	Copy active tab content to clipboard
Pobierz plik	Download single file from active tab

6.2 Export Tabs

Tab	File	Contents
config.json	`{name}-emitter.json`	Current emitter configuration for @pixi/particle-emitter
vfx-emitter.js	`{name}-vfx.js`	Ready-to-paste PixiJS integration function
vfx-states.json	`{name}-vfx-states.json`	All 5 win states with params + configs
timeline.json	`{name}-timeline.json`	Timeline keyframes + sampled config patches

7. ZIP Package Structure

The ZIP archive is production-ready for handing off to a game client team.

```
{symbol}-vfx/
├── README.txt
├── manifest.json
├── config/
│   ├── {symbol}-emitter.json
│   ├── {symbol}-params.json
│   ├── {symbol}-vfx-states.json
│   ├── {symbol}-timeline.json
│   └── {symbol}-emission-mask.json   (if mask ≠ point)
├── scripts/
│   └── {symbol}-vfx-emitter.js
└── textures/
```

```
    ├── particle.png                (active particle sprite)
    └── symbol.jpg                    (reference symbol image)
```

JSON configs reference textures with relative paths (`../textures/particle.png`) so they work inside the extracted folder structure.

Integrating in PixiJS

1. Load `textures/particle.png` as a `PIXI.Texture` .
2. Import or copy `scripts/{symbol}-vfx-emitter.js` .
3. Call `initSymbolVFX(container, texture, app.ticker)` .
4. Switch configs per win state using the states JSON or your game logic.

8. Recommended Workflow

1. Clone repo and start dev server in Cursor (`npm run dev`).
2. Upload or select a slot symbol — wait for Visual Agent analysis.
3. Preview each win state in the Win State Composer; tune parameters per state.
4. Set emission mask mode (outline/fill/hotspots) if particles should follow symbol shape.
5. Choose particle texture (Auto, Library, or Custom).
6. Adjust colors, curves, and motion until the preview matches art direction.
7. Build a timeline for animated win sequences if needed.
8. Download **ZIP Package** and deliver to the game integration team.

> **Pro tip:** Edit each win state separately before exporting. The ZIP includes all five states when analysis has been run at least once.

9. Troubleshooting

UI changes not visible after update

> **Stale dev server:** Run `npm run dev:restart` and hard-refresh the browser (`Cmd+Shift+R` / `Ctrl+Shift+R`). The app uses fixed port **5176**.

Port already in use

```
npm run dev:restart
```

This kills any process on port 5176 and starts a fresh Vite instance.

ZIP download fails

- Ensure a symbol is loaded (required for symbol texture in the archive).
- If using Custom particle texture, confirm the sprite uploaded successfully.
- Check the Visual Agent log for error messages after clicking ZIP.

Particles not visible

- Verify blend mode — try **add** on dark backgrounds.
- Increase Max Particles and check Spawn Interval.
- Confirm particle texture source is not empty (Custom requires upload).

Git push permission denied

Ensure your GitHub account has write access to `stefanskytom/vfx_editor`, or push using credentials for the repository owner account.